Yleaf Manual

Version 4.0

Yleaf manual

Version 4.0

Arwin Ralf, Diego Montiel Gonzalez, Kaiyin Zhong and Manfred Kayser

Department of Genetic Identification, Erasmus MC University Medical Centre Rotterdam

Contents

1 Installation

Yleaf runs on Linux, macOS, and Windows. There are three installation options: a standalone executable (recommended for most users), a conda environment (recommended for developers), and a manual install.

Yleaf requires Python 3.7 or higher for the source install options. The standalone executable bundles its own Python runtime and requires no Python installation.

1.1 Option 1: Standalone executable (recommended)

Pre-built binaries for all three platforms are available on the GitHub releases page. These bundles include Yleaf, samtools, bcftools, and minimap2 — no external tools need to be installed separately.

Download the archive for your platform:

- `yleaf-linux.tar.gz` — Linux (x86-64)
- `yleaf-macos.tar.gz` — macOS (Intel and Apple Silicon)
- `yleaf-windows.zip` — Windows (x86-64)

Linux / macOS:

```
tar xzf yleaf-linux.tar.gz
./yleaf/yleaf -h
```

Windows (PowerShell):

```
Expand-Archive yleaf-windows.zip
.\yleaf\yleaf.exe -h
```

1.2 Option 2: Conda environment (recommended for source installs)

```
# Clone the repository
git clone https://github.com/genid/Yleaf.git
cd Yleaf

# Create the conda environment (it will be called yleaf)
conda env create --file environment_yleaf.yaml

# Activate the environment
conda activate yleaf

# Install Yleaf into the environment
# The -e flag lets you edit config.txt in place without reinstalling
pip install -e .

# Verify the installation
Yleaf -h
```

Note: You must run `conda activate yleaf` each time you open a new terminal before using Yleaf.

1.3 Option 3: Manual install

```
# Install Python 3.7+ and required libraries
apt-get install python3
pip3 install pandas numpy

# Install external tools
sudo apt-get install minimap2 samtools bcftools

# Clone the Yleaf repository
git clone https://github.com/genid/Yleaf.git
cd Yleaf
pip install -e .

# Verify the installation
Yleaf -h
```

1.4 Reference genome configuration

After installation, navigate to `yleaf/config.txt` to add custom paths for the reference genomes. This prevents Yleaf from downloading them on the first run. Positions are based on hg38, hg19, or T2T (hs1). If no paths are configured, the reference genome will be downloaded automatically on the first run (approximately 3 GB per build).

2 Dashboard (GUI)

Yleaf 4.0 includes a graphical user interface (GUI) called the **Yleaf Dashboard**. This is the recommended way to use Yleaf for users who are not comfortable with the command line. The dashboard provides all the same functionality as the command-line interface in an interactive, point-and-click environment.

2.1 Launching the dashboard

From the standalone installer: Download the platform-specific dashboard installer from the GitHub releases page and install it like any other application:

- Linux: `.AppImage` (double-click to run, no install needed) or `.deb` (install via `sudo dpkg -i`)
- macOS: `.dmg` (drag to Applications)
- Windows: `.exe` or `.msi` installer

From source:

```
cd dashboard
npm install
npm run tauri dev
```

2.2 Dashboard features

The dashboard provides a complete graphical interface for:

- Loading BAM, CRAM, VCF, FASTQ, or PLINK files
- Selecting the reference genome and haplogroup tree(s)
- Running predictions with configurable thresholds
- Viewing per-sample haplogroup predictions and QC scores
- Inspecting the interactive haplogroup tree visualization
- Running and viewing forensic mixture analysis results

The Yleaf CLI backend runs as a bundled subprocess inside the dashboard — all analysis happens locally on your computer, with no data sent to external servers.

3 Command-line usage

3.1 Command-line options

All command-line options listed here can also be viewed by running `Yleaf -h` or `Yleaf --help`.

Input options (one required):

`-fastq PATH, --fastq PATH` Use raw FASTQ files as input. Yleaf will align the reads to the reference genome using minimap2.

`-bam PATH, --bamfile PATH` Input BAM file. Accepts a single file or a directory of BAM files.

`-cram PATH, --cramfile PATH` Input CRAM file.

`-vcf PATH, --vcffile PATH` Input VCF file (must be bgzip-compressed: `.vcf.gz`). The VCF must contain per-allele read counts in the `FORMAT/AD` field (produced by callers such as DeepVariant or GATK HaplotypeCaller).

`-plink PATH, --plinkfile PATH` PLINK format SNP array input. Pass the `.ped` file (text format) or the `.bed` file (binary format); companion files (`.map`/`.bim`/`.fam`) must be in the same directory. Both single- and multi-sample files are supported. Chromosome Y must be encoded as 24, `Y`, or `chrY`.

Reference genome:

`-rg {hg19,hg38,t2t}, --reference_genome {hg19,hg38,t2t}` *(required)* The reference genome build to use. Accepted values: `hg19`, `hg38`, `t2t` (T2T-CHM13v2.0 / hs1). If no reference is found at the configured path it will be downloaded automatically (~3 GB).

Output:

`-o STRING, --output STRING` *(required)* Output folder name. All result files for the run are written here.

Haplogroup tree selection:

`-tree {yfull,yfull_v10,ftdna,isogg} [...]` One or more haplogroup trees to use for prediction. Multiple trees can be specified separated by spaces (e.g. `-tree yfull ftdna`). When multiple trees are provided, a single combined pileup is built and prediction is run independently for each tree. Accepted values:

- `yfull` — YFull v14.01 tree (default)
- `yfull_v10` — YFull v10.01 legacy tree
- `ftdna` — FamilyTreeDNA Y-haplotree (all reference builds)
- `isogg` — ISOGG legacy tree

Quality filters:

`-r INT, --reads_treshold INT` Minimum number of reads required at each position for a base call to be made. (default: 10)

`-q INT, --quality_thresh INT` Minimum mapping quality for each read, integer between 10 and 40. (default: 20)

`-b INT, --base_majority INT` Minimum percentage of reads that must support the called base for it to be accepted, integer between 50 and 99. (default: 90)

`-pq FLOAT, --prediction_quality FLOAT` Minimum combined QC score (product of QC-1, QC-2, QC-3) required for a haplogroup prediction to be accepted. Range: 0–1. (default: 0.95)

Performance:

-t INT, --threads INT Number of parallel processes to use. (default: 1)

Special modes:

-aDNA, --ancient_DNA Enable ancient DNA mode. Ignores all G>A and C>T substitutions, which are common post-mortem deamination artefacts that would otherwise be misinterpreted as derived alleles.

-mix, --mixture Enable forensic mixture mode. Detects multiple Y-chromosome contributors in a mixed DNA sample, estimates the mixture ratio of each contributor, and assigns a haplogroup to each contributor. Requires per-allele read counts (BAM/CRAM input, or VCF with `FORMAT/AD`). When `-mix` is set, the base majority threshold is automatically raised to 99% unless explicitly overridden with `-b`.

Visualization:

-dh, --draw_haplogroups Generate an interactive HTML file showing all predicted haplogroups positioned in the haplogroup tree. The file contains per-haplogroup tabs, a zoomable SVG tree, and a PDF export button. Output file: `hg_tree_image.html` (or `haplogroup_tree_{tree}.html` in multi-tree mode).

-hc, --collapsed_draw_mode Use together with `-dh`. Compresses the tree visualization by removing haplogroup nodes that have no predicted samples, making large trees easier to navigate.

Private mutations:

-p, --private_mutations Search for private mutations: variants that are not part of any annotated haplogroup in the tree. These can be used to further differentiate individuals within the same haplogroup. Produces a `.pmu` file.

-maf FLOAT, --minor_allele_frequency FLOAT Maximum population allele frequency for a variant to be reported as a private mutation (from dbSNP). Variants with a higher frequency are excluded. (default: 0.01)

Workflow control:

-rp, --reuse_pileup If a pileup file (`.pu`) already exists in the output folder, skip the `samtools mpileup` step and reuse it. The `.pu` file is kept on disk when this flag is set. Useful for re-running predictions with different thresholds without repeating the (potentially long) pileup step.

-ra, --reanalyze For VCF input only. Skip the sorting and filtering step and re-use previously split VCF files.

-force, --force Delete existing output files without prompting for confirmation.

-cr PATH, --cram_reference PATH Path to the full reference genome FASTA file used for CRAM decompression. For CRAM files, the reference is usually auto-detected from the CRAM header; this flag is only needed if auto-detection fails.

3.2 Usage examples

BAM or CRAM:

```
Yleaf -bam file.bam -o output -rg hg38
Yleaf -cram file.cram -o output -rg hg38
```

FASTQ (raw reads):

```
Yleaf -fastq raw_reads.fastq -o output -rg hg38
```

VCF:

```
Yleaf -vcf variants.vcf.gz -o output -rg hg38
```

PLINK / SNP array:

```
Yleaf -plink dataset.bed -o output -rg hg38
```

Multiple haplogroup trees in one run:

```
Yleaf -bam file.bam -o output -rg hg38 -tree yfull ftdna isogg
```

With interactive tree visualization and private mutations:

```
Yleaf -bam file.bam -o output -rg hg38 -dh -p
```

Ancient DNA:

```
Yleaf -bam ancient_sample.bam -o output -rg hg38 -aDNA
```

Forensic mixture analysis:

```
Yleaf -bam mixture.bam -o output -rg hg38 -mix
```

Reuse a previously computed pileup (e.g., to tune thresholds):

```
Yleaf -bam file.bam -o existing_output -rg hg38 -rp -r 5 -b 80
```

Directory of BAM files (batch mode):

```
Yleaf -bam /path/to/bam_folder/ -o batch_output -rg hg38 -t 8
```

4 Output file formats

Each sample produces its own subdirectory inside the output folder. The subdirectory is named after the input file. In single-tree mode, file extensions are as described below. In multi-tree mode, per-tree files carry the tree name as an additional suffix (e.g., `.yfull.out`, `.ftdna.out`).

4.1 `.out` file

The main per-sample genotyping output. A tab-separated file with one row per marker that passed all quality filters. Columns:

Column	Description
`chr`	Chromosome (always chrY or Y)
`pos`	Genomic position (in the reference build used)
`marker_name`	Marker identifier from the positions file
`haplogroup`	Haplogroup node this marker defines
`mutation`	Base change at this position (e.g., A→G)
`anc`	Ancestral allele
`der`	Derived allele
`reads`	Number of reads at this position (after quality filtering)
`called_perc`	Percentage of reads supporting the called base
`called_base`	The base call that met the quality and majority thresholds
`state`	`A` = ancestral state called; `D` = derived state called
`depth`	Depth of this haplogroup node in the tree (higher = more specific)

4.2 `.fmf` file

The "failed markers" file. A tab-separated file with the same columns as the `.out` file, except `depth`, plus one additional column:

Column	Description
`Description`	Reason the marker did not pass quality filters. Possible values: `Position with zero reads`, `Below read threshold`, `Below base majority`, `Discordant genotype`

This file is useful for inspecting why specific markers were excluded and for tuning the `-r`, `-b`, and `-q` thresholds.

4.3 `.chr` file

A tab-separated file reporting the read distribution across all chromosomes in the alignment, generated using `samtools idxstats`. Only produced for BAM/CRAM/FASTQ input (not VCF or PLINK).

Column	Description
`chr`	Chromosome name from the alignment header

Column	Description
`reads`	Number of mapped reads on this chromosome
`perc`	Percentage of all mapped reads on this chromosome

This file is not used for prediction but is useful as a quality control check to verify the proportion of Y-chromosomal reads.

4.4 `.info` file

A plain-text file containing summary statistics for the run, including:

- Total mapped reads
- Total unmapped reads
- Total number of markers in the positions file
- Number of markers with zero reads
- Number of markers below the read threshold
- Number of markers below the base majority threshold
- Number of markers with discordant genotype
- Markers used for haplogroup prediction (valid markers)

In multi-tree mode a separate `.{tree}.info` file is written for each tree.

4.5 `.hg` file

The haplogroup prediction output. A tab-separated file with one row per sample. In single-tree mode, this file is named `hg_prediction.hg`. In multi-tree mode, a per-tree file `hg_prediction_{tree}.hg` and a combined file `hg_prediction_combined.hg` are written.

Single-tree `.hg` columns:

Column	Description
`Sample_name`	Sample identifier (input filename without extension)
`Hg`	Predicted haplogroup with sub-haplogroup exclusions (e.g., `E-Z15929*(xE-Y25504)`)
`Hg_marker`	Defining markers for the predicted haplogroup, separated by `;`
`Total_reads`	Total number of mapped reads for this sample
`Valid_markers`	Number of markers used in the prediction
`QC-score`	Overall prediction quality score (product of QC-1, QC-2, QC-3). Predictions below the `-pq` threshold (default 0.95) are not reported.
`QC-1`	Backbone score: whether the predicted haplogroup follows the expected major branch structure of the tree
`QC-2`	Equivalent-marker score: fraction of markers defining the final predicted haplogroup that are in the derived state
`QC-3`	Within-haplogroup score: whether all ancestor haplogroups along the path are also in the expected derived state

Combined multi-tree `.hg` columns:

The combined file contains one row per sample and one set of four columns per tree: `{tree}_Hg`, `{tree}_Hg_marker`, `{tree}_Valid_markers`, `{tree}_QC`.

Interpreting the QC scores:

QC-1 checks whether the predicted haplogroup belongs to the correct major branch. For example, if haplogroup E is predicted, the backbone nodes A0-T, A1, A1b, BT, CT, DE should all be in the derived state, while major branches CF, F, GHIJK, etc. should be ancestral. A score of 1 indicates all backbone nodes are in the expected state. If the score falls below the overall QC threshold, it is strongly recommended to manually inspect the `.out` file.

QC-2 checks whether the markers that specifically define the predicted haplogroup are in the derived state. For example, if the final prediction is R1b1a2 (R-Z2103), and there are two markers defining this haplogroup (Z2103 and Z2105), QC-2 = 1.0 if both are derived, 0.5 if one is derived and one is ancestral. A QC-2 below the overall threshold will cause the algorithm to fall back to the parental haplogroup.

QC-3 checks whether all intermediate ancestor haplogroups along the path from the root to the predicted haplogroup also have the expected (derived) state for their defining markers. A score of 1 indicates full consistency with the tree structure.

4.6 `.pmu` file

Generated when the `-p` (private mutations) flag is used. A tab-separated file listing variants that are not part of any annotated haplogroup in the tree, ordered by dbSNP annotation status.

Column	Description
`chrom`	Chromosome (always Y)
`position`	Genomic position
`rn_no`	dbSNP rs number (if available; `-` if not annotated)
`mutation`	Observed mutation (e.g., `A->G`)
`reference`	Reference (major) allele
`detected`	Detected (minor) allele
`reads`	Number of reads supporting the detected allele
`called_percentage`	Percentage of reads supporting the detected allele
`minor allele frequency`	Population allele frequency from dbSNP (if available; `NA` otherwise)

4.7 `.mix` file

Generated when the `-mix` (mixture) flag is used and a mixture signal is detected. A tab-separated file describing the contributing Y-chromosome haplogroups in the mixed sample.

Header lines (prefixed with `#`):

Header	Description
`#mixture_version`	File format version
`#tree`	Haplogroup tree used for this analysis

Header	Description
`#common_ancestor`	Deepest shared ancestor of all detected contributors
`#n_contributors`	Number of detected contributors

Data columns:

Column	Description
`contributor`	Contributor rank (1 = largest contributor)
`haplogroup`	Predicted haplogroup of this contributor, with sub-haplogroup exclusions (e.g., `E-M96*(xE-V38,E-M35)`)
`ratio`	Estimated DNA mixture ratio for this contributor (fraction, 0–1)
`markers`	Number of heterozygous positions supporting this contributor's branch
`qc`	QC score for this contributor's haplogroup assignment

In multi-tree mode, a separate `.{tree}.mix` file is written for each tree.

Note on VCF input: Mixture analysis from VCF requires the `FORMAT/AD` field. VCFs produced by `bcftools mpileup` without the `-a AD` flag do not contain allelic depth and cannot be used for mixture analysis.

4.8 .pu file (pileup)

The intermediate pileup file produced by `samtools mpileup`, named `temp_haplogroup_pileup.pu`. By default this file is deleted after use. When the `-rp` flag is set, it is retained so that subsequent runs can skip the pileup step entirely.

4.9 hg_tree_image.html

Generated when the `-dh` flag is used. A self-contained interactive HTML file showing all predicted haplogroups positioned in the haplogroup tree. Features include:

- Separate tab per major haplogroup branch
- Zoomable and pannable SVG tree rendering
- Per-haplogroup node highlighting for predicted samples
- In-browser PDF export

In multi-tree mode, a separate `haplogroup_tree_{tree}.html` file is written for each tree.

5 Version changes

Yleaf version 4.0 (May 2026):

- Added graphical dashboard (GUI) for Windows, macOS, and Linux — the primary interface for end users
- Added support for PLINK/SNP-array input (`-plink`)
- Added multi-tree prediction: run all supported trees in a single command with one shared pileup (`-tree yfull ftdna isogg`)
- Added FamilyTreeDNA (FTDNA) haplogroup tree support with hg19, hg38, and T2T positions
- Added ISOGG haplogroup tree support
- Added YFull v14.01 as the new default tree (replacing v10.01; ~235,000 marker positions on hg38)
- Added forensic mixture analysis mode (`-mix`): detects multiple Y-chromosome contributors, estimates ratios, and assigns a haplogroup per contributor
- Added ancient DNA mode (`-aDNA`): ignores G>A and C>T post-mortem deamination artefacts
- Added pileup reuse flag (`-rp`): skip mpileup if a `.pu` file already exists
- Added T2T (CHM13v2.0 / hs1) reference genome support
- Replaced static PDF haplogroup tree output with an interactive self-contained HTML file (`-dh`)
- Added VCF missing-position inference: positions absent from the VCF are inferred from the reference genome, restoring ~99% of markers
- Standalone executables for Linux, macOS, and Windows bundling samtools, bcftools, and minimap2 (no external tool installation required)
- Removed the legacy `-old` prediction mode (ISOGG v2.3 only)

Yleaf version 3.2 (January 2024):

- Included support for VCF files
- Optimized the prediction algorithm by adapting the QC-1 value to the YFull tree
- Added the `-cr` option for CRAM files
- Added the `-pq` option to define the prediction quality threshold
- Increased the default `-pq` to 0.95, making the prediction more stringent
- Resolved compatibility issues when using the `-old` option

Yleaf version 3.1 (November 2022):

- Added an option to find private mutations (`-p`)
- Added an option to draw a haplogroup tree showing the relation between samples (`-dh`)
- Simplified command-line input: only the reference genome needs to be specified; all data files are located automatically
- Reference data files are downloaded on first run (no separate install step required)
- Yleaf can now be called from any directory after pip install

Yleaf version 3.0 (April 2022):

- Haplogroup tree is now based on YFull(v10.01) and ISOGG data
- Tree has been updated with new information from both YFull(v10.01) and ISOGG
- A haplogroup can now be ancestral, derived or undefined; undefined states (fewer than 60% of markers in either direction) are excluded from QC scoring

Yleaf version 2.2 (March 2020):

- Minor bug fixes in haplogroup prediction algorithm
- Minor bug fixes in the genotyping algorithm
- Included CRAM format as supported input
- Included position files for two targeted genotyping assays

Yleaf version 2.1 (December 2019):

- Bug fix in the haplogroup prediction algorithm

Before version 2.0:

- Upgraded marker list with bugs from Yleaf v2.0 fixed (based on ISOGG, July 2019)
- Moved from Python 2.7 to Python 3.6
- Replaced R code with Python code, resulting in a significant increase in run speed

6 Contact information

For questions about the interpretation of results and general feedback about the tool, please contact:

a.ralf@erasmusmc.nl

To report a bug or ask about installation, please open an issue on GitHub:

https://github.com/genid/Yleaf/issues